EXPT NO: 3	TEXT GENERATION USING N-GRAM LANGUAGE MODEL
DATE: 25/02/26	

AIM

To implement a Generative Adversarial Network (GAN) to generate the synthetic handwritten digit images using the MNIST dataset.

ALGORITHM/ STEPS

- **STEP 1: Load Dataset**

  Load MNIST handwritten digit images and normalize them.

- **STEP 2: Build Generator**

  Create a neural network that:
  - Takes random noise as input
  - Generates fake images

- **STEP 3: Build Discriminator**

  Create a neural network that:
  - Takes image as input
  - Classifies real vs fake

- **STEP 4: Adversarial Training**

  Repeat for multiple epochs:
  - Train discriminator on real and fake images
  - Train generator to fool discriminator

- **STEP 5: Generate Images**

  Use trained generator to produce synthetic digits.

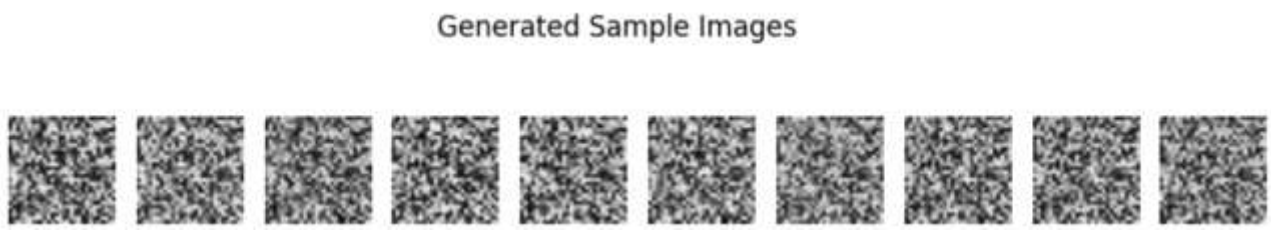
CODE IMPLEMENTATION

```
import tensorflow as tf
import numpy as np
from tensorflow.keras import layers

# -------- Load MNIST (small subset) --------
(x_train, _), _ = tf.keras.datasets.mnist.load_data()

#x_train = x_train[:5000] # use small data
x_train = (x_train - 127.5) / 127.5
x_train = x_train.reshape(-1, 784)

# Generator
generator = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(100,)),
    layers.Dense(784, activation='tanh')
])

# -------- Discriminator --------
discriminator = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(1, activation='sigmoid')
])

discriminator.compile(optimizer='adam', loss='binary_crossentropy')

# GAN Model
discriminator.trainable = False
gan_input = tf.keras.Input(shape=(100,))
fake_img = generator(gan_input)
gan_output = discriminator(fake_img)

gan = tf.keras.Model(gan_input, gan_output)
gan.compile(optimizer='adam', loss='binary_crossentropy')

# Training
batch_size = 64

for epoch in range(50): # very small for exam
    idx = np.random.randint(0, x_train.shape[0], batch_size)
    real_imgs = x_train[idx]
    noise = np.random.normal(0, 1, (batch_size, 100))
    fake_imgs = generator.predict(noise, verbose=0)

    # Train Discriminator
    d_loss_real = discriminator.train_on_batch(real_imgs,
np.ones((batch_size,1)))
```

```
    d_loss_fake = discriminator.train_on_batch(fake_imgs,
np.zeros((batch_size,1)))

    # Train Generator
    noise = np.random.normal(0, 1, (batch_size, 100))
    g_loss = gan.train_on_batch(noise, np.ones((batch_size,1)))
    print(f"Epoch {epoch+1} | D Loss: {d_loss_real:.4f} | G Loss:
{g_loss:.4f}")

print("GAN Training Completed")
import matplotlib.pyplot as plt

# Generate random noise for new images
num_samples = 10 # Number of sample images to generate
noise = np.random.normal(0, 1, (num_samples, 100))

# Generate images from the noise using the generator
generated_images = generator.predict(noise)

# Reshape images to 28x28 and rescale to [0, 1] for display
generated_images = generated_images.reshape(num_samples, 28, 28)
generated_images = (generated_images + 1) / 2.0 # Rescale from [-1, 1] to [0,
1]

# Plot the generated images
plt.figure(figsize=(10, 2))
for i in range(num_samples):
    plt.subplot(1, num_samples, i + 1)
    plt.imshow(generated_images[i], cmap='gray')
    plt.axis('off')
plt.suptitle('Generated Sample Images')
plt.show()
```

OUTPUT

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/trainer.py:86: UserWarning: The model does not have any trainable weights.
  warnings.warn("The model does not have any trainable weights.")
Epoch 1 | D Loss: 0.8071 | G Loss: 0.7790
Epoch 2 | D Loss: 0.7405 | G Loss: 0.5861
Epoch 3 | D Loss: 0.8211 | G Loss: 0.4587
Epoch 4 | D Loss: 0.9535 | G Loss: 0.3691
Epoch 5 | D Loss: 1.0982 | G Loss: 0.3055
Epoch 6 | D Loss: 1.2414 | G Loss: 0.2593
Epoch 7 | D Loss: 1.4124 | G Loss: 0.2248
Epoch 8 | D Loss: 1.5691 | G Loss: 0.1978
Epoch 9 | D Loss: 1.7077 | G Loss: 0.1765
Epoch 10 | D Loss: 1.8520 | G Loss: 0.1592
Epoch 11 | D Loss: 1.9977 | G Loss: 0.1450
Epoch 12 | D Loss: 2.1344 | G Loss: 0.1331
Epoch 13 | D Loss: 2.2568 | G Loss: 0.1230
Epoch 14 | D Loss: 2.3707 | G Loss: 0.1143
Epoch 15 | D Loss: 2.4706 | G Loss: 0.1067
Epoch 16 | D Loss: 2.5799 | G Loss: 0.1001
Epoch 17 | D Loss: 2.6790 | G Loss: 0.0942
Epoch 18 | D Loss: 2.7768 | G Loss: 0.0890
Epoch 19 | D Loss: 2.8620 | G Loss: 0.0844
Epoch 20 | D Loss: 2.9457 | G Loss: 0.0802
Epoch 21 | D Loss: 3.0285 | G Loss: 0.0763
Epoch 22 | D Loss: 3.1035 | G Loss: 0.0729
Epoch 23 | D Loss: 3.1789 | G Loss: 0.0697
Epoch 24 | D Loss: 3.2478 | G Loss: 0.0668
Epoch 25 | D Loss: 3.3125 | G Loss: 0.0642
Epoch 26 | D Loss: 3.3769 | G Loss: 0.0617
Epoch 27 | D Loss: 3.4366 | G Loss: 0.0594
Epoch 28 | D Loss: 3.4927 | G Loss: 0.0573
Epoch 29 | D Loss: 3.5454 | G Loss: 0.0553
Epoch 30 | D Loss: 3.5967 | G Loss: 0.0535

Epoch 31 | D Loss: 3.6449 | G Loss: 0.0518
Epoch 32 | D Loss: 3.6884 | G Loss: 0.0502
Epoch 33 | D Loss: 3.7319 | G Loss: 0.0486
Epoch 34 | D Loss: 3.7720 | G Loss: 0.0472
Epoch 35 | D Loss: 3.8128 | G Loss: 0.0459
Epoch 36 | D Loss: 3.8498 | G Loss: 0.0446
Epoch 37 | D Loss: 3.8855 | G Loss: 0.0434
Epoch 38 | D Loss: 3.9193 | G Loss: 0.0423
Epoch 39 | D Loss: 3.9536 | G Loss: 0.0412
Epoch 40 | D Loss: 3.9842 | G Loss: 0.0402
Epoch 41 | D Loss: 4.0153 | G Loss: 0.0392
Epoch 42 | D Loss: 4.0475 | G Loss: 0.0382
Epoch 43 | D Loss: 4.0741 | G Loss: 0.0374
Epoch 44 | D Loss: 4.0980 | G Loss: 0.0365
Epoch 45 | D Loss: 4.1279 | G Loss: 0.0357
Epoch 46 | D Loss: 4.1521 | G Loss: 0.0349
Epoch 47 | D Loss: 4.1775 | G Loss: 0.0342
Epoch 48 | D Loss: 4.1998 | G Loss: 0.0335
Epoch 49 | D Loss: 4.2216 | G Loss: 0.0328
Epoch 50 | D Loss: 4.2451 | G Loss: 0.0321
GAN Training Completed
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 96ms/step
```



RESULT:

Upon The Completion of this experiment, The GAN model was successfully trained on the MNIST dataset. The generator learned to produce synthetic handwritten digit images by competing against the discriminator.